DOCUMENTATION TECHNIQUE · VERSION 2.0.0

Jira Test Case Generator

Des critères d'acceptation aux cas de test, par IA locale.

Architecture · API REST · Sécurité · Tests · Exploitation

POINTS CLÉS

IA 100 % locale

37 tests automatisés

API typée

AUTEUR	Alex Alkhatib
STACK	Java 17 · Spring Boot 3.5.16 · Spring AI 1.1.8
MODÈLE	Ollama (local, mistral par défaut)
DATE	9 novembre 2026
LICENCE	MIT

Sommaire

1. Résumé exécutif . . . 3

2. Contexte et objectifs . . . 4

2.1 Le besoin . . . 4

2.2 Objectifs de la refonte . . . 4

2.3 Périmètre . . . 4

3. Audit de la version 1 . . . 5

4. Architecture cible . . . 6

4.1 Composants principaux . . . 6

5. Flux de génération . . . 7

5.1 Exemple de requête . . . 7

5.2 Extrait de réponse . . . 7

6. Stack technique . . . 8

7. Interface web . . . 9

7.1 Fonctions de l'interface . . . 9

8. Référence API . . . 10

8.1 Paramètres . . . 10

8.2 Codes d'erreur . . . 10

9. Sécurité et configuration . . . 11

9.1 Mesures appliquées . . . 11

9.2 Variables d'environnement . . . 11

10. Qualité et tests . . . 12

10.1 Vérification manuelle . . . 12

10.2 Limites de la validation . . . 12

11. Exploitation et déploiement . . . 13

11.1 Démarrage local . . . 13

11.2 JAR autonome . . . 13

11.3 Docker Compose . . . 13

11.4 Supervision . . . 13

12. Performance et limites . . . 14

12.1 Limites connues . . . 14

13. Feuille de route . . . 15

13.1 Conclusion . . . 15

14. Annexes . . . 16

14.1 Structure du projet . . . 16

14.2 Commandes utiles . . . 16

14.3 Glossaire . . . 16

1. Résumé exécutif

La version 1 du **Jira Test Case Generator** démontrait une idée prometteuse : lire une user story Jira et confier à un modèle local la rédaction de cas de test. Elle n'était cependant pas exploitable en l'état : un **token Jira était commité en clair**, la dépendance Spring AI pointait vers un instantané de développement aujourd'hui introuvable, la réponse de l'API était du texte libre, et aucun test ne couvrait le comportement.

La version 2.0.0 est une **réécriture complète** qui conserve le cœur fonctionnel — Spring Boot, Jira, Ollama — et ajoute ce qui manquait pour un outil réel : un contrat d'API typé et documenté, une extraction robuste des critères d'acceptation, une gestion d'erreurs normalisée, une interface web, une image Docker, une configuration sans secret, et **37 tests automatisés** dont un test d'intégration HTTP de bout en bout.

Indicateur	Valeur
Tests automatisés	37 (unitaires + intégration HTTP)
Dépendances obsolètes ou SNAPSHOT	0
Secrets présents dans le dépôt	0
Statuts HTTP distincts gérés	400 · 404 · 502 · 503 · 500
Formats d'export	JSON · CSV · Markdown
Temps de démarrage mesuré	3,8 s (JAR, JVM 17)
Génération mesurée (stub local)	232 ms pour 6 cas

Point d'attention critique

Le token Atlassian présent dans l'archive d'origine doit être considéré comme compromis. Il doit être révoqué depuis la console Atlassian puis remplacé par un token frais fourni uniquement par variable d'environnement. La version 2 ne contient plus aucun secret.

2. Contexte et objectifs

2.1 Le besoin

Dans une équipe Agile, une story Jira doit être transformée en plan de test exécutable. Ce travail est répétitif : identifier le chemin nominal, imaginer les cas d'erreur, les valeurs limites, les risques de sécurité et d'accessibilité, puis rédiger des étapes reproductibles. La qualité du résultat dépend fortement de la disponibilité et de l'expérience du testeur.

L'automatisation visée n'est pas de remplacer le QA, mais de produire une **première trame complète** que le testeur valide, ajuste et enrichit. Le modèle doit rester **local** : les descriptions Jira contiennent souvent des informations métier sensibles.

2.2 Objectifs de la refonte

- **Fiabilité** : construire et démarrer sans dépendance à un snapshot, sans secret dans le code.
- **Contrat clair** : renvoyer du JSON typé, exploitable par une UI, un script ou un pipeline CI.
- **Robustesse** : couvrir les cas Jira Cloud et Server, les descriptions ADF, les réponses IA imparfaites.
- **Observabilité** : erreurs normalisées, logs utiles, sonde de santé, endpoint de configuration.
- **Expérience** : une interface web simple, des exports, un historique local.
- **Qualité** : des tests qui protègent réellement le comportement, exécutables sans Jira ni GPU.

2.3 Périmètre

Le périmètre couvre la lecture d'une story unique par clé, la génération de cas de test et leur export. Ne sont volontairement pas inclus : la persistance en base, l'écriture dans Jira/Xray, l'authentification multi-utilisateurs et l'exécution des tests. Ces sujets sont traités dans la feuille de route en section 13.

3. Audit de la version 1

L'audit a porté sur les 16 fichiers de l'archive d'origine. Les constats ci-dessous sont classés par impact décroissant. Chaque ligne de la colonne « traitement » renvoie à une décision d'architecture concrète de la version 2.

#	Constat	Sévérité	Traitement en v2
1	Token API Jira et email en clair dans application.properties	Critique	Supprimés du dépôt, variables d'environnement, .env ignoré, rotation documentée
2	Dépendance spring-ai 1.0.0-SNAPSHOT + dépôts snapshot	Critique	Spring Boot 3.5.16 + Spring AI 1.1.8 (BOM stable, versions épinglées)
3	Aucun test de comportement (contextLoads uniquement)	Élevée	37 tests : parseurs, client Jira, prompts, service, intégration HTTP
4	Réponse texte brut, HTTP 200 même en erreur	Élevée	DTO JSON + ProblemDetail RFC 7807 + statuts 400/404/502/503/500
5	Description ADF Jira Cloud interprétée comme du texte	Élevée	AdfTextExtractor : paragraphes, titres, listes, sauts de ligne
6	Aucune validation de la clé de story	Moyenne	Regex PROJET-123, normalisation, bornes sur maxCases
7	exception avalée, message générique sans diagnostic	Moyenne	Exceptions métier typées, codes stables, logs ciblés, messages actionnables
8	Démarrage impossible sans credentials (validation en bean)	Moyenne	Démarrage tolérant, état exposé par /api/v1/config, 503 à l'usage
9	Dépendances Jira client/Jersey/Jackson redondantes	Moyenne	RestClient Spring, timeouts explicites, aucune dépendance superflue
10	defaultFunctions déprécié, API Spring AI obsolète	Moyenne	ChatClient moderne, aucun appel déprécié
11	Sortie IA non contrainte, non parsable	Moyenne	Contrat JSON strict, parseur tolérant, erreur AI_RESPONSE_INVALID
12	Aucune protection contre l'injection de prompt	Faible	Contexte délimité, troncature, consigne « données non fiables »
13	Pas d'interface, pas de packaging, pas de documentation	Faible	UI responsive, Dockerfile, compose, README, PDF, présentation

4. Architecture cible

Le code est organisé par responsabilité : **web** (contrat HTTP), **service** (orchestration), **jira** (intégration distante), **ai** (prompting et parsing), **config** (propriétés), **error** (exceptions métier). Les dépendances vont toujours de l'extérieur vers l'intérieur.

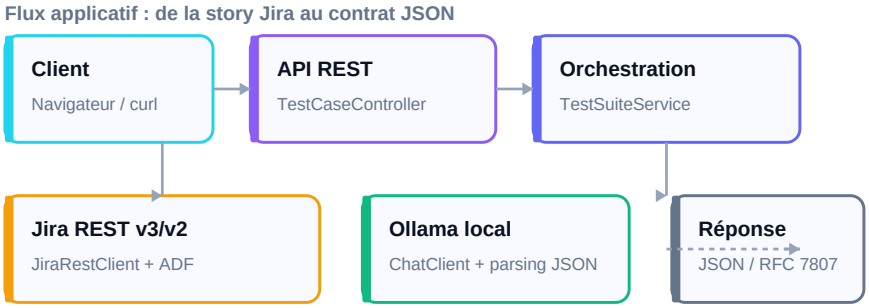


Figure 1 — Flux applicatif de la requête HTTP jusqu'à la réponse structurée.

4.1 Composants principaux

Composant	Rôle	Décision notable
TestCaseController	Expose /api/v1/test-cases	Alias /jira-story conservé pour la compatibilité
TestSuiteService	Valide, orchestre, mappe	Aucune logique HTTP dans les couches basses
JiraRestClient	Lit l'issue distante	RestClient, Basic auth, timeouts, ADF + v2
AcceptanceCriteriaExtractor	Isole les critères	Section FR/EN, inline, champ custom, fallback
PromptFactory	Construit les prompts	Garde-fou anti-injection, troncature, contrat JSON
TestCaseGenerationService	Appelle le modèle	Bornes, avertissements, mesure de durée
TestCaseResponseParser	Parse la sortie IA	Fences, alias, prose, entrées partielles
ApiExceptionHandler	Normalise les erreurs	RFC 7807, code stable, pas de fuite interne

5. Flux de génération

Une requête suit huit étapes déterministes. Chaque étape peut échouer avec un code distinct, ce qui permet à l'appelant de réagir précisément sans lire un message libre.

Étape	Action	Échec possible
1	Validation de storyKey, langue, format, maxCases	VALIDATION_ERROR (400)
2	Lecture de l'issue Jira avec Basic auth et timeout	STORY_NOT_FOUND (404), JIRA_AUTH_FAILED (502)
3	Aplatissement de la description ADF ou texte	JIRA_UNAVAILABLE (502)
4	Extraction des critères d'acceptation	Repli documenté sur la description
5	Construction du prompt borné et sécurisé	—
6	Appel du modèle Ollama local	AI_UNAVAILABLE (503)
7	Parsing JSON tolérant et troncature	AI_RESPONSE_INVALID (502)
8	Sérialisation du DTO de réponse	INTERNAL_ERROR (500)

5.1 Exemple de requête

```
curl "http://localhost:8080/api/v1/test-cases?storyKey=KAN-3&language=FR&format=STANDARD&maxCases=6"
```

5.2 Extrait de réponse

```
{
  "story": {
    "key": "KAN-3",
    "summary": "Connexion utilisateur",
    "acceptanceCriteriaSource": "description-section"
  },
  "generation": { "model": "mistral", "language": "FR", "durationMs": 8421 },
  "testCases": [
    { "id": "TC-001", "title": "Connexion réussie", "type": "Fonctionnel",
      "priority": "Haute", "steps": ["Ouvrir la page", "Saisir les identifiants"],
      "expectedResult": "Le tableau de bord s'affiche" }
  ],
  "warnings": []
}
```

Pourquoi un contrat JSON strict ?

Un LLM local produit facilement des réponses légèrement différentes d'un appel à l'autre. En imposant un schéma et en parsant la sortie de façon tolérante, l'application transforme une sortie probabiliste en données déterministes : l'interface, les exports et les tests deviennent stables.

6. Stack technique

Composant	Version	Rôle et justification
Java	17 (Temurin)	LTS, compatibilité Spring Boot 3.x
Spring Boot	3.5.16	Web, validation, actuator, packaging exécutable
Spring AI	1.1.8	Abstraction ChatClient et connecteur Ollama, version stable
Gradle	8.8 (wrapper)	Build reproductible, aucun outil global requis
Jira REST API	v3 Cloud / v2 Server	Lecture des issues, ADF ou texte brut
Ollama	courant	Inférence locale, aucune donnée envoyée à un tiers
JUnit 5 / AssertJ	via Boot BOM	Tests unitaires expressifs
MockWebServer	4.12.0	Simulation HTTP de Jira, tests sans tenant
HTML / CSS / JS	natif	Interface sans étape de build ni dépendance CDN
Docker	multi-stage	Image JRE non-root, healthcheck

Le choix de supprimer le client Jira officiel et Jersey mérite une explication : l'application n'utilise qu'un endpoint, `GET /rest/api/{version}/issue/{key}`. Le client officiel apporte un graphe de dépendances important, des conflits Jakarta et une surface d'API inutile. Spring `RestClient` couvre le besoin en quelques lignes, avec des timeouts explicites et un contrôle total sur les erreurs.

7. Interface web

L'interface est servie directement par Spring Boot depuis `src/main/resources/static`. Elle ne dépend d'aucun CDN, d'aucune police distante et d'aucun build front-end : elle fonctionne dans un réseau isolé, ce qui est cohérent avec le positionnement « IA locale ».

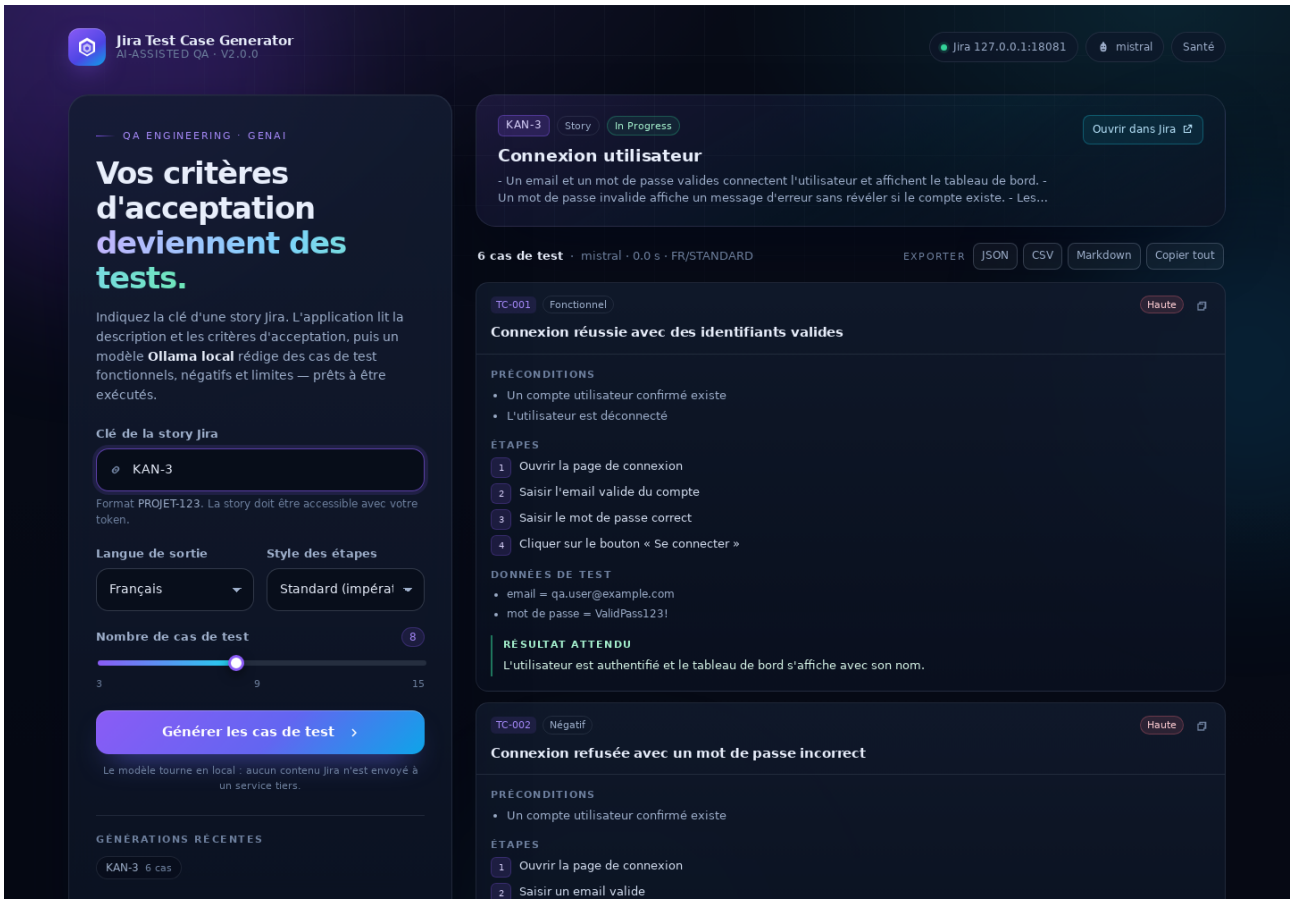


Figure 2 — Vue des résultats : story, métadonnées de génération, cas de test et exports.

7.1 Fonctions de l'interface

- Formulaire validé côté client et côté serveur (clé, langue, format, nombre de cas).
- Indicateurs de configuration Jira et de modèle, bannière explicite si Jira n'est pas configuré.
- Rendu des cas en cartes : type, priorité, préconditions, étapes numérotées, données, résultat attendu.
- Exports JSON, CSV (séparateur « ; », BOM UTF-8 pour Excel) et Markdown ; copie d'un cas ou de tous.
- Historique des huit dernières générations, stocké uniquement dans le navigateur.
- Gestion des états : chargement avec chronomètre, erreurs actionnables, avertissements non bloquants.
- Accessibilité : libellés, focus visible, navigation clavier, respect de prefers-reduced-motion.

8. Référence API

Méthode	Chemin	Description
GET	/api/v1/test-cases	Génère les cas de test d'une story
GET	/api/v1/jira-story	Alias rétrocompatible de l'endpoint précédent
GET	/api/v1/config	Configuration publique, sans secret
GET	/actuator/health	Sonde de santé (Docker, Kubernetes)
GET	/	Interface web

8.1 Paramètres

Paramètre	Requis	Défaut	Valeurs
storyKey	Oui	—	PROJET-123
language	Non	FR	FR, EN
format	Non	STANDARD	STANDARD, GHERKIN
maxCases	Non	8	1 à 20 (configurable)

8.2 Codes d'erreur

Code	HTTP	Cause	Action recommandée
VALIDATION_ERROR	400	Paramètre absent ou invalide	Corriger la requête
STORY_NOT_FOUND	404	Clé inconnue ou projet non autorisé	Vérifier la clé et les droits
JIRA_AUTH_FAILED	502	401/403 renvoyé par Jira	Régénérer le token Atlassian
JIRA_UNAVAILABLE	502	Timeout, 5xx, réponse illisible	Vérifier URL, réseau, version d'API
JIRA_NOT_CONFIGURED	503	Variables JIRA_* absentes	Compléter la configuration
AI_UNAVAILABLE	503	Ollama arrêté ou modèle absent	ollama serve puis ollama pull
AI_RESPONSE_INVALID	502	Sortie du modèle non exploitable	Réessayer ou changer de modèle
INTERNAL_ERROR	500	Erreur inattendue	Consulter les logs serveur

9. Sécurité et configuration

À faire immédiatement : révoquer le token exposé

L'archive d'origine contenait un email et un token API Jira en clair dans application.properties. Même si le dépôt est privé, considérez ce token comme compromis : révoquez-le depuis id.atlassian.com, générez-en un nouveau, et ne le placez que dans un fichier .env local ou un gestionnaire de secrets (Vault, AWS Secrets Manager, GitHub Actions Secrets).

9.1 Mesures appliquées

- Aucune credential dans le code, les tests, les ressources ou les logs.
- Fichier `.env` ignoré par Git ; `.env.example` fournit le contrat.
- `/api/v1/config` n'expose que des indicateurs (booléen, URL, version).
- Validation des entrées avant tout appel distant ; bornes sur la taille et le nombre.
- Timeouts de connexion et de lecture côté Jira pour éviter l'épuisement de threads.
- Contexte story délimité et tronqué ; consigne explicite de ne jamais suivre ses instructions.
- Rendu de l'interface via textContent, jamais innerHTML avec des données distantes.
- Exceptions internes journalisées côté serveur, messages client sans détail sensible.
- Conteneur Docker non-root, healthcheck, aucune socket ou volume inutile.

9.2 Variables d'environnement

Variable	Défaut	Rôle
JIRA_BASE_URL	—	URL de l'instance, avec ou sans /rest/api/3
JIRA_EMAIL	—	Compte Atlassian (Basic auth)
JIRA_API_TOKEN	—	Token API, jamais commité
JIRA_API_VERSION	3	3 = Cloud, 2 = Server/Data Center
JIRA_ACCEPTANCE_CRITERIA_FIELD	—	Champ custom des critères d'acceptation
JIRA_TIMEOUT	15s	Connect et read timeout
OLLAMA_BASE_URL	http://localhost:11434	Endpoint Ollama
AI_MODEL	mistral	Modèle utilisé
AI_TEMPERATURE	0.2	Créativité du modèle
GENERATION_MAX_TEST_CASES	20	Plafond dur de cas par requête

L'application démarre volontairement sans credentials. Cela permet de valider l'installation, d'afficher un diagnostic clair et de déployer l'image avant de brancher les secrets. L'erreur n'apparaît qu'au moment de l'usage, avec un code 503 et la liste des variables manquantes.

10. Qualité et tests

La suite compte **37 tests**, exécutés avec `./gradlew test`. Le test d'intégration démarre le contexte Spring complet sur un port aléatoire, expose les vrais endpoints HTTP, remplace Jira par MockWebServer et le modèle par un ChatModel factice. Aucun tenant Atlassian, aucun GPU et aucune connexion externe ne sont nécessaires.

Suite	Tests	Ce qu'elle protège
AdfTextExtractorTest	4	ADF Cloud, texte v2, nœuds inconnus, null
AcceptanceCriteriaExtractorTest	6	Sections FR/EN, inline, fallback, section vide
JiraRestClientTest	7	Mapping, Basic auth, champ custom, 404/401/500, URL legacy
TestCaseResponseParserTest	6	Fences, tableau nu, alias, prose, entrées invalides
PromptFactoryTest	4	Contenu, Gherkin/EN, troncature, critères absents
TestCaseGenerationServiceTest	6	Limites, avertissements, erreurs IA
ApiIntegrationTest	8	Contrat HTTP, erreurs, config, UI, health
Total	37	Tous verts au 9 novembre 2026

10.1 Vérification manuelle

Au-delà des tests automatisés, l'application a été lancée contre un Jira simulé et un Ollama simulé, puis pilotée dans un navigateur Chromium en 1440 × 1000. La génération a produit 6 cas en 232 ms, les exports JSON, CSV et Markdown ont été déclenchés, et les parcours d'erreur 404 et 401 ont été observés. Les captures de ce document proviennent de cette exécution.

10.2 Limites de la validation

- Aucun tenant Jira réel ni modèle Ollama réel n'était disponible dans l'environnement de build ; les intégrations ont été validées contre des serveurs simulant les contrats HTTP.
- La qualité rédactionnelle des cas dépend du modèle choisi : mistral est un bon compromis, llama3.1 ou qwen2.5 produisent souvent un JSON plus fiable.
- Aucun test de charge n'a été réalisé : le goulot d'étranglement est l'inférence locale, non l'application.

```
./gradlew test --console=plain
# 37 tests, 0 échec — BUILD SUCCESSFUL
```

11. Exploitation et déploiement

11.1 Démarrage local

```
cp .env.example .env          # renseigner JIRA_*
ollama pull mistral
./gradlew bootRun             # http://localhost:8080
```

11.2 JAR autonome

```
./gradlew clean bootJar
java -jar build/libs/jira-test-case-generator.jar
```

11.3 Docker Compose

```
cp .env.example .env
docker compose up -d --build
docker compose exec ollama ollama pull mistral
```

L'image est construite en deux étapes (Gradle puis JRE Alpine), s'exécute avec un utilisateur non privilégié et expose un healthcheck sur `/actuator/health`. Le modèle est téléchargé explicitement : cette séparation évite de re-télécharger plusieurs gigaoctets à chaque redémarrage et rend le démarrage prévisible.

11.4 Supervision

- Sonde de disponibilité : `GET /actuator/health` (statut UP/DOWN).
- Version déployée : `GET /actuator/info` et `/api/v1/config`.
- Logs : niveau INFO pour l'application, WARN pour Spring AI ; aucun token ni contenu intégral.
- Erreurs : chaque échec porte un code stable, facile à agréger dans un tableau de bord.

> **Recommandation de production**
>
> Placez l'application derrière un reverse proxy TLS et une authentification (OIDC ou mTLS). Elle ne gère volontairement pas les comptes utilisateurs : c'est un outil interne, pas un service public. Restreignez également l'accès réseau à Jira et à Ollama.

12. Performance et limites

Mesure	Valeur observée	Contexte
Démarrage de l'application	3,8 s	JAR, JVM 17, contexte Spring complet
Génération HTTP de bout en bout	232 ms	Jira et Ollama simulés, 6 cas
Taille du JAR	~45 Mo	Spring Boot + Spring AI, modèle exclu
Mémoire JVM	~250 Mo	MaxRAMPercentage fixé à 75 % en Docker
Cas maximum par requête	20	Configurable, borne dure
Contexte story maximum	8 000 caractères	Troncature explicite et signalée

Avec un vrai modèle local, la latence est dominée par l'inférence : comptez de 10 à 60 secondes pour 8 cas avec mistral sur un CPU récent, moins avec un GPU. L'application n'ajoute qu'un aller-retour Jira et un parsing JSON. Les timeouts HTTP sont réglés à 15 secondes pour Jira et 3 minutes pour le client Ollama afin d'absorber les générations longues.

12.1 Limites connues

- Un seul modèle et une seule story par requête ; pas de génération asynchrone.
- Pas de cache : une même story est relue et régénérée à chaque appel.
- Le parsing tolérant masque parfois une sortie partielle du modèle ; les avertissements le signalent.
- Les critères d'acceptation heuristiques peuvent inclure du contexte superflu si la story est mal structurée.
- Aucune persistance : l'historique vit dans le navigateur et disparaît avec lui.

13. Feuille de route

Priorité	Évolution	Valeur attendue
P1	Export Xray / Zephyr (CSV d'import dédié)	Supprimer la ressaisie dans l'outil de test
P1	Cache des stories Jira avec TTL	Réduire la latence et les appels API
P2	Génération en streaming (SSE)	Afficher les cas au fil de la réponse
P2	Persistance PostgreSQL des suites	Historique partagé et traçabilité
P2	Lien direct story / suite de tests	Traçabilité QA et audit
P3	Authentification SSO et quotas	Ouvrir l'outil à plusieurs équipes
P3	Comparaison multi-modèles	Choisir le modèle le plus pertinent
P3	Génération asynchrone pour stories longues	Éviter les timeouts navigateur

13.1 Conclusion

La version 2 transforme une démonstration en outil utilisable : elle démarre sans secret, expose un contrat stable, tolère les imperfections d'un LLM local, affiche un diagnostic précis et se déploie en une commande. Les 37 tests et la vérification navigateur donnent une base saine pour les évolutions à venir, en particulier l'export vers les outils de gestion de tests.

14. Annexes

14.1 Structure du projet

```
jira-test-case-generator/
■■■ build.gradle, settings.gradle, gradle.properties
■■■ Dockerfile, docker-compose.yml, .env.example
■■■ README.md, LICENSE.md
■■■ docs/                  # ce document, la présentation, les scripts
■■■ src/
    ■■■ main/java/com/asecurityguru/jiratestgen/
    ■   ■■■ ai/ jira/ service/ web/ config/ error/
    ■■■ main/resources/
        ■■■ application.yml
        ■■■ static/          # index.html, css, js, favicon
    ■■■ test/java/           # 37 tests
```

14.2 Commandes utiles

Commande	Usage
./gradlew test	Exécuter les 37 tests
./gradlew bootRun	Lancer en développement
./gradlew clean bootJar	Construire le JAR exécutable
docker compose up -d --build	Démarrer l'application et Ollama
curl localhost:8080/api/v1/config	Vérifier la configuration détectée
curl localhost:8080/actuator/health	Vérifier la disponibilité

14.3 Glossaire

Terme	Définition
ADF	Atlassian Document Format : arbre JSON utilisé par Jira Cloud pour le texte enrichi.
Ollama	Serveur d'inférence local exposant une API HTTP compatible avec plusieurs modèles.
Critère d'acceptation	Condition observable qui définit la réussite d'une story.
RFC 7807	Standard des réponses d'erreur HTTP (application/problem+json).
Gherkin	Syntaxe Given/When/Then utilisée par les outils BDD.
Prompt injection	Tentative de faire exécuter des instructions malveillantes via des données.

Jira Test Case Generator 2.0.0 — document généré le 09/11/2026 avec reportlab. Sources : dépôt du projet.